Courtify

Sports Arena Booking Platform

Sprint 1 – Test Cases

Project	Courtify – Sports Arena Booking Platform
Sprint	1
Prepared By	Shaheer, Yousuf, Zarrar, Wasiq
Date	March 31, 2026
Version	1.0

Test Environment

Parameter	Value
Frontend	http://localhost:5173
Backend	http://localhost:5000
Database	MySQL – courtify_db
Browser	Google Chrome (latest)
API Tool	Postman
Postman-test samples	/postman/collections/Courtify Sprint 1 Tests

Module 1 – User Registration

TC-REG-001 – Successful player registration with valid inputs	
Test Case ID	TC-REG-001
Module	User Registration
Title	Successful player registration with valid inputs
Priority	High
Pre-conditions	Backend running; email "newplayer@test.com" not in DB
Test Steps	1. Go to signup page. 2. Select "Player". 3. Fill all fields. 4. Click Sign Up.
Test Data	Name: Ali Khan, Email: newplayer@test.com, Phone: 03001234567, Password: Pass@1234
Expected Result	HTTP 201; "Account created. Check your email to verify." shown; record in players table with is_active=0
Actual Result	HTTP 201 returned; message shown on UI; DB record confirmed
Status	**Pass**
Comments	Verification token generated and stored correctly

TC-REG-002 – Successful arena owner registration with valid inputs	
Test Case ID	TC-REG-002
Module	User Registration
Title	Successful arena owner registration with valid inputs
Priority	High
Pre-conditions	Backend running; email "newowner@test.com" not in DB
Test Steps	1. Go to signup page. 2. Select "Owner". 3. Fill all fields. 4. Click Sign Up.
Test Data	Name: Sara Malik, Email: newowner@test.com, Phone: 03009876543, Password: Owner@5678
Expected Result	HTTP 201; record in arena_owners with is_active=0; verification email sent
Actual Result	HTTP 201; DB record confirmed; email dispatched via Nodemailer
Status	**Pass**
Comments	Separate arena_owners table used correctly

TC-REG-003 – Registration fails with duplicate player email	
Test Case ID	TC-REG-003
Module	User Registration
Title	Registration fails with duplicate player email
Priority	High
Pre-conditions	Email “newplayer@test.com” already exists in players table
Test Steps	1. Go to signup. 2. Select “Player”. 3. Enter existing email. 4. Click Sign Up.
Test Data	Email: newplayer@test.com, Password: AnyPass123
Expected Result	HTTP 409; error message “Email already registered”
Actual Result	HTTP 409; correct error shown on UI
Status	**Pass**
Comments	Duplicate check applied before insert

TC-REG-004 – Registration fails with duplicate owner email	
Test Case ID	TC-REG-004
Module	User Registration
Title	Registration fails with duplicate owner email
Priority	High
Pre-conditions	Email “newowner@test.com” already exists in arena_owners
Test Steps	1. Go to signup. 2. Select “Owner”. 3. Enter existing email. 4. Click Sign Up.
Test Data	Email: newowner@test.com, Password: AnyPass123
Expected Result	HTTP 409; “Email already registered”
Actual Result	HTTP 409; error shown correctly
Status	**Pass**
Comments	Duplicate check works for owner table separately

TC-REG-005 – Registration fails with invalid email format	
Test Case ID	TC-REG-005
Module	User Registration
Title	Registration fails with invalid email format
Priority	High
Pre-conditions	Backend running
Test Steps	1. Go to signup. 2. Enter malformed email. 3. Click Sign Up.
Test Data	Email: notanemail, Password: Pass@1234
Expected Result	HTTP 400; “Invalid email format”
Actual Result	HTTP 400; correct error
Status	**Pass**
Comments	Regex validation applied server-side

TC-REG-006 – Registration fails when email is missing	
Test Case ID	TC-REG-006
Module	User Registration
Title	Registration fails when email is missing
Priority	High
Pre-conditions	Backend running
Test Steps	1. Send POST /auth/signup without email field.
Test Data	Body: { “password”: “Pass@1234”, “userType”: “player” }
Expected Result	HTTP 400; “Email and password required”
Actual Result	HTTP 400; correct error
Status	**Pass**
Comments	Server-side guard before processing

TC-REG-007 – Registration fails when password is missing	
Test Case ID	TC-REG-007
Module	User Registration
Title	Registration fails when password is missing
Priority	High
Pre-conditions	Backend running
Test Steps	1. Send POST /auth/signup without password field.
Test Data	Body: { “email”: “x@test.com”, “userType”: “player” }
Expected Result	HTTP 400; “Email and password required”
Actual Result	HTTP 400; correct error
Status	**Pass**
Comments	Same guard catches missing password

TC-REG-008 – Registration fails with invalid userType	
Test Case ID	TC-REG-008
Module	User Registration
Title	Registration fails with invalid userType
Priority	Medium
Pre-conditions	Backend running
Test Steps	1. Send POST /auth/signup with userType = “admin”.
Test Data	Body: { “email”: “x@test.com”, “password”: “Pass@1234”, “userType”: “admin” }
Expected Result	HTTP 400; “Invalid user type”
Actual Result	HTTP 400; correct error
Status	**Pass**
Comments	Only “player” and “owner” are accepted

TC-REG-009 – Password is stored as bcrypt hash, not plain text	
Test Case ID	TC-REG-009
Module	User Registration
Title	Password is stored as bcrypt hash, not plain text
Priority	High
Pre-conditions	Successful registration completed
Test Steps	1. Register user. 2. Query players table for the record. 3. Check password_hash field.
Test Data	Email: checkpass@test.com, Password: PlainPass99
Expected Result	password_hash starts with "$2b$10$" (bcrypt format); not equal to "PlainPass99"
Actual Result	Hash confirmed starting with $2b$10$; plain text not stored
Status	**Pass**
Comments	Salt rounds = 10 used

Module 2 – Email Verification

TC-VER-001 – Player account activated via valid verification link	
Test Case ID	TC-VER-001
Module	Email Verification
Title	Player account activated via valid verification link
Priority	High
Pre-conditions	Player registered; is_active=0; valid token in players table; email received
Test Steps	1. Open verification email. 2. Click the verification link. 3. Observe redirect. 4. Check DB.
Test Data	Token: valid 64-char hex token from DB
Expected Result	HTTP 302 redirect to /?verified=1&type=player; is_active=1; verification_token=NULL in DB
Actual Result	Redirect confirmed; DB updated correctly
Status	**Pass**
Comments	Both player and owner share same endpoint, tried player first

TC-VER-002 – Owner account activated via valid verification link	
Test Case ID	TC-VER-002
Module	Email Verification
Title	Owner account activated via valid verification link
Priority	High
Pre-conditions	Owner registered; is_active=0; valid token in arena_owners table
Test Steps	1. Click verification link from owner email. 2. Check redirect. 3. Check DB.
Test Data	Token: valid 64-char hex token from arena_owners
Expected Result	Redirect to /?verified=1&type=owner; is_active=1; token cleared
Actual Result	Owner redirect confirmed; DB updated
Status	**Pass**
Comments	Fallback to arena_owners after player check finds no match

TC-VER-003 – Verification fails with invalid token	
Test Case ID	TC-VER-003
Module	Email Verification
Title	Verification fails with invalid token
Priority	High
Pre-conditions	Backend running
Test Steps	1. Send GET /auth/verify?token=invalidtoken123
Test Data	token: invalidtoken123
Expected Result	HTTP 400; “Invalid or expired token”
Actual Result	HTTP 400; correct message returned
Status	**Pass**
Comments	No DB modification occurs

TC-VER-004 – Verification fails when token parameter is absent	
Test Case ID	TC-VER-004
Module	Email Verification
Title	Verification fails when token parameter is absent
Priority	Medium
Pre-conditions	Backend running
Test Steps	1. Send GET /auth/verify (no token param)
Test Data	No query parameters
Expected Result	HTTP 400; “Invalid verification link”
Actual Result	HTTP 400; correct message
Status	**Pass**
Comments	Early return before DB query

TC-VER-005 – Reusing an already-used verification token fails	
Test Case ID	TC-VER-005
Module	Email Verification
Title	Reusing an already-used verification token fails
Priority	High
Pre-conditions	User already verified; verification_token = NULL in DB
Test Steps	1. Click the same verification link a second time.
Test Data	Token: same token used in TC-VER-001
Expected Result	HTTP 400; "Invalid or expired token"; no DB changes
Actual Result	HTTP 400; token already NULL; account untouched
Status	**Pass**
Comments	Token cleared on first use – prevents replay

TC-VER-006 – Dashboard shows success banner after verification redirect	
Test Case ID	TC-VER-006
Module	Email Verification
Title	Dashboard shows success banner after verification redirect
Priority	Medium
Pre-conditions	User clicks verification link and is redirected to /?verified=1&type=player
Test Steps	1. Complete verification. 2. Observe dashboard page.
Test Data	URL: http://localhost:5173/?verified=1&type=player
Expected Result	Success message displayed (e.g. "Email verified! You can now log in.")
Actual Result	Success banner shown correctly on dashboard
Status	**Pass**
Comments	Query param read on component mount

TC-VER-007 – Verification email contains correctly formatted link	
Test Case ID	TC-VER-007
Module	Email Verification
Title	Verification email contains correctly formatted link
Priority	High
Pre-conditions	User registers successfully
Test Steps	1. Register new player. 2. Open received email. 3. Inspect the link URL.
Test Data	Email: linkcheck@test.com
Expected Result	Link format: http://localhost:5000/auth/verify?token=¡64-char-hex¿
Actual Result	Link matches expected format; clickable button present in HTML email
Status	**Pass**
Comments	Email uses styled HTML template

TC-VER-008 – Signup page shows “check your email” message after registration	
Test Case ID	TC-VER-008
Module	Email Verification
Title	Signup page shows “check your email” message after registration
Priority	Medium
Pre-conditions	Backend running; Nodemailer configured
Test Steps	1. Complete signup form. 2. Submit. 3. Observe UI.
Test Data	Valid new player credentials
Expected Result	Frontend displays message prompting user to check their email
Actual Result	“Check your email to verify” shown after signup
Status	**Pass**
Comments	Message shown regardless of email delivery success

Module 3 – User Login

TC-LOG-001 – Verified player logs in successfully	
Test Case ID	TC-LOG-001
Module	User Login
Title	Verified player logs in successfully
Priority	High
Pre-conditions	Player account in DB; is_active=1
Test Steps	1. Go to login. 2. Select "Player". 3. Enter credentials. 4. Click Login.
Test Data	Email: player@example.com, Password: 12345678
Expected Result	HTTP 200; user data returned; redirected to dashboard; AuthContext updated
Actual Result	HTTP 200; Dashboard loaded with user session
Status	**Pass**
Comments	Player dashboard shown; owner route not accessible

TC-LOG-002 – Verified owner logs in and is redirected to owner dashboard	
Test Case ID	TC-LOG-002
Module	User Login
Title	Verified owner logs in and is redirected to owner dashboard
Priority	High
Pre-conditions	Owner account in DB; is_active=1
Test Steps	1. Select "Owner" on login. 2. Enter credentials. 3. Click Login.
Test Data	Email: marksman_admin@example.com, Password: 12345678
Expected Result	HTTP 200; userType="owner"; redirect to /owner/dashboard
Actual Result	HTTP 200; Owner dashboard loaded
Status	**Pass**
Comments	isOwner() check in AuthContext drives the redirect

TC-LOG-003 – Unverified player cannot log in	
Test Case ID	TC-LOG-003
Module	User Login
Title	Unverified player cannot log in
Priority	High
Pre-conditions	Player registered; is_active=0 (email not verified)
Test Steps	1. Enter credentials of unverified player. 2. Click Login.
Test Data	Email: unverified@test.com, Password: Pass@1234
Expected Result	HTTP 403; "Please verify your email first"; no session created
Actual Result	HTTP 403; error shown on login UI
Status	**Pass**
Comments	is_active check happens before bcrypt comparison

TC-LOG-004 – Login fails with incorrect password	
Test Case ID	TC-LOG-004
Module	User Login
Title	Login fails with incorrect password
Priority	High
Pre-conditions	Verified player account exists
Test Steps	1. Enter correct email but wrong password. 2. Click Login.
Test Data	Email: player@example.com, Password: wrongpass
Expected Result	HTTP 401; "Invalid email or password"
Actual Result	HTTP 401; error message shown
Status	**Pass**
Comments	Generic message does not reveal whether email exists

TC-LOG-005 – Login fails with non-existent email	
Test Case ID	TC-LOG-005
Module	User Login
Title	Login fails with non-existent email
Priority	High
Pre-conditions	Email not present in DB
Test Steps	1. Enter unregistered email. 2. Enter any password. 3. Click Login.
Test Data	Email: ghost@nowhere.com, Password: AnyPass123
Expected Result	HTTP 401; "Invalid email or password"
Actual Result	HTTP 401; generic error (no email existence disclosure)
Status	**Pass**
Comments	Secure – does not confirm whether email is registered

TC-LOG-006 – Login fails with missing email parameter	
Test Case ID	TC-LOG-006
Module	User Login
Title	Login fails with missing email parameter
Priority	Medium
Pre-conditions	Backend running
Test Steps	1. Send GET /auth/validate without email param.
Test Data	Query: ?password=abc&userType=player
Expected Result	HTTP 400; "Email and password required"
Actual Result	HTTP 400; correct error
Status	**Pass**
Comments	Guards before DB query

TC-LOG-007 – Login fails with invalid userType	
Test Case ID	TC-LOG-007
Module	User Login
Title	Login fails with invalid userType
Priority	Medium
Pre-conditions	Backend running
Test Steps	1. Send GET /auth/validate with userType=superadmin.
Test Data	Email: player@example.com, Password: 12345678, userType: superadmin
Expected Result	HTTP 400; "Invalid user type"
Actual Result	HTTP 400; correct error
Status	**Pass**
Comments	Only "player" and "owner" accepted

TC-LOG-008 – Login response does not contain password or hash	
Test Case ID	TC-LOG-008
Module	User Login
Title	Login response does not contain password or hash
Priority	High
Pre-conditions	Verified player account exists
Test Steps	1. Log in successfully. 2. Inspect HTTP response body.
Test Data	Email: player@example.com, Password: 12345678
Expected Result	Response contains: authenticated, userId, email, name, userType – no password_hash field
Actual Result	Response inspected; no password or hash present
Status	**Pass**
Comments	Critical security check

TC-LOG-009 – Session persists on page refresh	
Test Case ID	TC-LOG-009
Module	User Login
Title	Session persists on page refresh
Priority	Medium
Pre-conditions	Player logged in successfully
Test Steps	1. Log in as player. 2. Refresh the browser page. 3. Observe NavBar and state.
Test Data	Email: player@example.com, Password: 12345678
Expected Result	User remains logged in; NavBar reflects authenticated state
Actual Result	Session persists via localStorage; NavBar shows logged-in state
Status	**Pass**
Comments	AuthContext reads from localStorage on mount

Module 4 – Arena and Court Listing

TC-LST-001 – All arenas returned with correct fields	
Test Case ID	TC-LST-001
Module	Arena Listing
Title	All arenas returned with correct fields
Priority	High
Pre-conditions	3 arenas seeded in DB with images
Test Steps	1. Send GET /arenas. 2. Inspect response.
Test Data	None (reads DB)
Expected Result	HTTP 200; array of 3 arenas with id, name, location, pricePerHour, availability, rating, image_path
Actual Result	All 3 arenas returned with correct fields
Status	**Pass**
Comments	image_path from subquery on arena_images table

TC-LST-002 – Arena listing renders VenueCards on dashboard	
Test Case ID	TC-LST-002
Module	Arena Listing
Title	Arena listing renders VenueCards on dashboard
Priority	High
Pre-conditions	Frontend and backend running
Test Steps	1. Open http://localhost:5173. 2. Observe arena cards.
Test Data	None
Expected Result	3 VenueCards displayed showing name, city, price, image
Actual Result	3 VenueCards rendered correctly
Status	**Pass**
Comments	Cards are clickable – navigate to venue detail

TC-LST-003 – Venue detail page shows correct data for Legends Arena	
Test Case ID	TC-LST-003
Module	Arena Detail
Title	Venue detail page shows correct data for Legends Arena
Priority	High
Pre-conditions	Legends Arena (id=1) seeded with full details
Test Steps	1. Send GET /arena/1. 2. Inspect response.
Test Data	Arena ID: 1
Expected Result	HTTP 200; name, address, city, rating, pricePerHour, timing, amenities (array), description, rules (array), images, courts
Actual Result	All fields returned correctly
Status	**Pass**
Comments	amenities and rules returned as parsed arrays, not raw JSON strings

TC-LST-004 – Legends Arena shows only Futsal and Padel courts (no Swimming Pool)	
Test Case ID	TC-LST-004
Module	Arena Detail – Error Correction
Title	Legends Arena shows only Futsal and Padel courts (no Swimming Pool)
Priority	High
Pre-conditions	Legends Arena seeded with Futsal Court and Padel Court only
Test Steps	1. Open venue detail for Legends Arena. 2. Check courts section.
Test Data	Arena ID: 1
Expected Result	courts object contains only “Futsal” and “Padel” keys; no “Swimming Pool”
Actual Result	Only Futsal and Padel shown; no incorrect types
Status	**Pass**
Comments	Core bug-fix validation – sport type driven by DB join, not hardcoded

TC-LST-005 – Marksman Arena shows correct court types	
Test Case ID	TC-LST-005
Module	Arena Detail – Error Correction
Title	Marksman Arena shows correct court types
Priority	High
Pre-conditions	Marksman Arena (id=2) seeded with Badminton and Futsal courts
Test Steps	1. Open venue detail for Marksman Arena. 2. Check courts section.
Test Data	Arena ID: 2
Expected Result	Courts section shows “Badminton” and “Futsal” only
Actual Result	Correct types displayed; no phantom sports
Status	**Pass**
Comments	Regression test for the court-type fix

TC-LST-006 – Titan Arena shows correct court types	
Test Case ID	TC-LST-006
Module	Arena Detail – Error Correction
Title	Titan Arena shows correct court types
Priority	High
Pre-conditions	Titan Arena (id=3) seeded with Padel and Tennis courts
Test Steps	1. Open venue detail for Titan Arena. 2. Check courts section.
Test Data	Arena ID: 3
Expected Result	Courts section shows “Padel” and “Tennis” only
Actual Result	Correct types shown
Status	**Pass**
Comments	Titan Arena has availability=closed; still shows detail

TC-LST-007 – Closed arena appears in listing with correct availability status	
Test Case ID	TC-LST-007
Module	Arena Listing
Title	Closed arena appears in listing with correct availability status
Priority	Medium
Pre-conditions	Titan Arena seeded with availability = 'closed'
Test Steps	1. Load dashboard. 2. Find Titan Arena card. 3. Check availability field.
Test Data	Arena ID: 3
Expected Result	Titan Arena visible; availability = "closed" in API response; card marked accordingly
Actual Result	Arena card present; availability field correct
Status	**Pass**
Comments	Closed arenas are not filtered out of listing

TC-LST-008 – Amenities returned as parsed array, not JSON string	
Test Case ID	TC-LST-008
Module	Arena Detail
Title	Amenities returned as parsed array, not JSON string
Priority	High
Pre-conditions	Legends Arena has JSON amenities in DB
Test Steps	1. Send GET /arena/1. 2. Check amenities field type.
Test Data	Arena ID: 1
Expected Result	amenities is a JS array e.g. ["Changing Rooms", "Showers", "Parking", "Equipment Rental"]
Actual Result	Amenities returned as parsed array
Status	**Pass**
Comments	safeJSON() handles Buffer/string cases from MySQL

TC-LST-009 – Rules returned as parsed array, not JSON string	
Test Case ID	TC-LST-009
Module	Arena Detail
Title	Rules returned as parsed array, not JSON string
Priority	High
Pre-conditions	Legends Arena has JSON rules in DB
Test Steps	1. Send GET /arena/1. 2. Check rules field type.
Test Data	Arena ID: 1
Expected Result	rules is a JS array e.g. [“Proper sports shoes required”, ...]
Actual Result	Rules returned as parsed array
Status	**Pass**
Comments	Same safeJSON() handles both fields

TC-LST-010 – Non-existent arena returns 404	
Test Case ID	TC-LST-010
Module	Arena Detail
Title	Non-existent arena returns 404
Priority	Medium
Pre-conditions	Backend running; arena ID 99999 does not exist
Test Steps	1. Send GET /arena/99999.
Test Data	Arena ID: 99999
Expected Result	HTTP 404; “Arena not found”
Actual Result	HTTP 404 returned
Status	**Pass**
Comments	Result array length check before responding

TC-LST-011 – Arenas returned in descending ID order	
Test Case ID	TC-LST-011
Module	Arena Listing
Title	Arenas returned in descending ID order
Priority	Low
Pre-conditions	Multiple arenas in DB
Test Steps	1. Send GET /arenas. 2. Check order of returned arenas.
Test Data	None
Expected Result	Arenas returned with highest ID first (ORDER BY id DESC)
Actual Result	Newest arenas appear first in response
Status	**Pass**
Comments	Useful for showing recently added arenas first

TC-LST-012 – Courts grouped by type in venue detail response	
Test Case ID	TC-LST-012
Module	Arena Detail
Title	Courts grouped by type in venue detail response
Priority	High
Pre-conditions	Legends Arena has Futsal and Padel courts seeded
Test Steps	1. Send GET /arena/1. 2. Check courts field structure.
Test Data	Arena ID: 1
Expected Result	courts is an object: { “Futsal”: [{ id, name }], “Padel”: [{ id, name }] }
Actual Result	Grouped court object returned correctly
Status	**Pass**
Comments	Grouping done in application layer using forEach

Module 5 – Search and Filter

TC-SCH-001 – Search by arena name filters results in real-time	
Test Case ID	TC-SCH-001
Module	Search and Filter
Title	Search by arena name filters results in real-time
Priority	Medium
Pre-conditions	3 arenas visible on dashboard
Test Steps	1. Type “Legend” in search bar. 2. Observe cards.
Test Data	Search input: “Legend”
Expected Result	Only “Legends Arena” card visible; others hidden
Actual Result	Filter works; only matching card displayed
Status	**Pass**
Comments	No page reload; purely state-based

TC-SCH-002 – Search by city filters results in real-time	
Test Case ID	TC-SCH-002
Module	Search and Filter
Title	Search by city filters results in real-time
Priority	Medium
Pre-conditions	3 arenas (Karachi x2, Lahore x1) on dashboard
Test Steps	1. Type “Lahore” in search bar. 2. Observe cards.
Test Data	Search input: “Lahore”
Expected Result	Only Marksman Arena (Lahore) visible
Actual Result	Only Marksman shown; Karachi arenas hidden
Status	**Pass**
Comments	Case-insensitive match recommended for Sprint 2

TC-SCH-003 – Clearing search restores all arena cards	
Test Case ID	TC-SCH-003
Module	Search and Filter
Title	Clearing search restores all arena cards
Priority	Medium
Pre-conditions	User has typed in search bar
Test Steps	1. Enter “Legend” in search. 2. Clear the input. 3. Observe cards.
Test Data	Search input: “” (after clear)
Expected Result	All 3 arena cards visible again
Actual Result	All cards restored on clear
Status	**Pass**
Comments	Filter resets when input is empty

TC-SCH-004 – Search with no matching input shows empty state	
Test Case ID	TC-SCH-004
Module	Search and Filter
Title	Search with no matching input shows empty state
Priority	Medium
Pre-conditions	3 arenas on dashboard
Test Steps	1. Type “zzzznotexist” in search bar. 2. Observe results.
Test Data	Search input: “zzzznotexist”
Expected Result	No arena cards displayed; empty state or “No results” shown
Actual Result	No cards shown; empty state displayed
Status	**Pass**
Comments	UI should handle zero-result state gracefully

TC-SCH-005 – Search is case-insensitive for arena name	
Test Case ID	TC-SCH-005
Module	Search and Filter
Title	Search is case-insensitive for arena name
Priority	Low
Pre-conditions	3 arenas on dashboard
Test Steps	1. Type “legends” (lowercase) in search bar. 2. Observe cards.
Test Data	Search input: “legends”
Expected Result	Legends Arena still found despite lowercase
Actual Result	Match found – filter is case-insensitive
Status	**Pass**
Comments	toLowerCase() applied on both sides in filter logic

Module 6 – Owner: Arena Registration

TC-OWN-001 – Owner successfully creates a new arena	
Test Case ID	TC-OWN-001
Module	Owner Arena Registration
Title	Owner successfully creates a new arena
Priority	High
Pre-conditions	Logged in as owner; on /owner/register-facility page
Test Steps	1. Fill all form fields. 2. Submit form.
Test Data	Name: Sports Hub, City: Islamabad, Address: F-7 Markaz, Price: 3000, Timing: 8 AM - 10 PM
Expected Result	HTTP 201; "Arena created successfully" with new arena id; arena appears in owner dashboard
Actual Result	HTTP 201; arena created and visible in dashboard
Status	**Pass**
Comments	availability defaults to 'available'; rating defaults to 0

TC-OWN-002 – Arena creation fails with missing required fields	
Test Case ID	TC-OWN-002
Module	Owner Arena Registration
Title	Arena creation fails with missing required fields
Priority	High
Pre-conditions	Backend running
Test Steps	1. Send POST /arenas without required fields.
Test Data	Body: { "name": "Incomplete" }
Expected Result	HTTP 400; "Missing required fields"
Actual Result	HTTP 400; correct error
Status	**Pass**
Comments	owner_id, name, city, pricePerHour are required

TC-OWN-003 – Arena default values set correctly on creation	
Test Case ID	TC-OWN-003
Module	Owner Arena Registration
Title	Arena default values set correctly on creation
Priority	Medium
Pre-conditions	Owner creates arena with minimum required fields
Test Steps	1. Create arena with only required fields. 2. Query arenas table.
Test Data	owner_id: 1, name: Test, city: Karachi, pricePerHour: 2000
Expected Result	availability = 'available'; rating = 0.00
Actual Result	DB confirms availability='available' and rating=0.00
Status	**Pass**
Comments	Defaults set by SQL schema

TC-OWN-004 – Owner dashboard lists own arenas via GET /owner/arenas	
Test Case ID	TC-OWN-004
Module	Owner Arena Registration
Title	Owner dashboard lists own arenas via GET /owner/arenas
Priority	High
Pre-conditions	Owner logged in; has at least 1 arena in DB
Test Steps	1. Send GET /owner/arenas?ownerId=1. 2. Inspect response.
Test Data	ownerId: 1
Expected Result	HTTP 200; array of arenas belonging to owner 1; amenities and rules as parsed arrays
Actual Result	Correct arenas returned; JSON fields parsed
Status	**Pass**
Comments	amenities/rules parsed from JSON strings in response

TC-OWN-005 – GET /owner/arenas returns empty array when owner has no arenas	
Test Case ID	TC-OWN-005
Module	Owner Arena Registration
Title	GET /owner/arenas returns empty array when owner has no arenas
Priority	Medium
Pre-conditions	New owner with no registered arenas
Test Steps	1. Send GET /owner/arenas?ownerId=999.
Test Data	ownerId: 999 (no arenas)
Expected Result	HTTP 200; empty array []
Actual Result	HTTP 200; [] returned
Status	**Pass**
Comments	No error; just empty result set

TC-OWN-006 – GET /owner/arenas fails without ownerId param	
Test Case ID	TC-OWN-006
Module	Owner Arena Registration
Title	GET /owner/arenas fails without ownerId param
Priority	Medium
Pre-conditions	Backend running
Test Steps	1. Send GET /owner/arenas (no ownerId).
Test Data	No query params
Expected Result	HTTP 400; “Owner ID required”
Actual Result	HTTP 400; correct error
Status	**Pass**
Comments	Guard before DB query

Module 7 – Route Protection

TC-RTE-001 – Player cannot access /owner/dashboard	
Test Case ID	TC-RTE-001
Module	Route Protection
Title	Player cannot access /owner/dashboard
Priority	High
Pre-conditions	User logged in as Player (userType = "player")
Test Steps	1. Log in as player. 2. Manually navigate to /owner/dashboard.
Test Data	userType: player
Expected Result	Redirected to "/" (Dashboard); owner dashboard not shown
Actual Result	Player redirected to main dashboard
Status	**Pass**
Comments	isOwner() returns false; Navigate component fires

TC-RTE-002 – Player cannot access /owner/register-facility	
Test Case ID	TC-RTE-002
Module	Route Protection
Title	Player cannot access /owner/register-facility
Priority	High
Pre-conditions	User logged in as Player
Test Steps	1. Log in as player. 2. Navigate to /owner/register-facility.
Test Data	userType: player
Expected Result	Redirected to "/"
Actual Result	Redirected to main dashboard
Status	**Pass**
Comments	Same protection logic applied to both owner routes

TC-RTE-003 – Unauthenticated user cannot access owner routes	
Test Case ID	TC-RTE-003
Module	Route Protection
Title	Unauthenticated user cannot access owner routes
Priority	High
Pre-conditions	User is NOT logged in (no session)
Test Steps	1. Open browser fresh (no saved session). 2. Navigate to /owner/dashboard.
Test Data	No login state in localStorage
Expected Result	Redirected to "/"
Actual Result	User redirected to main dashboard
Status	**Pass**
Comments	isOwner() returns false when no user in context

TC-RTE-004 – Owner can access /owner/dashboard	
Test Case ID	TC-RTE-004
Module	Route Protection
Title	Owner can access /owner/dashboard
Priority	High
Pre-conditions	User logged in as Owner
Test Steps	1. Log in as owner. 2. Navigate to /owner/dashboard.
Test Data	userType: owner
Expected Result	Owner dashboard page loads correctly
Actual Result	Dashboard loads; arena list shown
Status	**Pass**
Comments	Positive test to confirm protection doesn't block valid owner

TC-RTE-005 – Owner can access /owner/register-facility	
Test Case ID	TC-RTE-005
Module	Route Protection
Title	Owner can access /owner/register-facility
Priority	High
Pre-conditions	User logged in as Owner
Test Steps	1. Log in as owner. 2. Navigate to /owner/register-facility.
Test Data	userType: owner
Expected Result	Facility registration form loads
Actual Result	Form displayed correctly
Status	**Pass**
Comments	Positive test for owner route access

Module 8 – Database Integrity

TC-DB-001 – Arena foreign key enforced – invalid owner_id rejected	
Test Case ID	TC-DB-001
Module	Database Integrity
Title	Arena foreign key enforced – invalid owner_id rejected
Priority	Medium
Pre-conditions	Backend running; owner ID 9999 does not exist
Test Steps	1. Send POST /arenas with owner_id=9999.
Test Data	owner_id: 9999, name: Test, city: Lahore, pricePerHour: 2000
Expected Result	HTTP 500 or 400; DB insert fails due to FK constraint
Actual Result	DB rejects insert; error returned
Status	**Pass**
Comments	arenas.owner_id FK references arena_owners.id

TC-DB-002 – Court FK enforced – invalid court_type_id rejected	
Test Case ID	TC-DB-002
Module	Database Integrity
Title	Court FK enforced – invalid court_type_id rejected
Priority	Medium
Pre-conditions	DB running
Test Steps	1. Attempt to insert a court with court_type_id=99 (doesn't exist).
Test Data	arena_id: 1, court_type_id: 99, name: Test Court
Expected Result	DB insert fails; FK violation error
Actual Result	Insert rejected by MySQL
Status	**Pass**
Comments	courts.court_type_id FK references court_types.id

TC-DB-003 – Player email uniqueness enforced at DB level	
Test Case ID	TC-DB-003
Module	Database Integrity
Title	Player email uniqueness enforced at DB level
Priority	High
Pre-conditions	Player with email “player@example.com” exists
Test Steps	1. Attempt direct DB insert with same email.
Test Data	email: player@example.com
Expected Result	MySQL UNIQUE constraint violation; insert rejected
Actual Result	Insert fails with duplicate entry error
Status	**Pass**
Comments	Application-level check also in place (TC-REG-003)

Test Execution Summary

Module	Total	Pass	Fail	Blocked
User Registration	9	9	0	0
Email Verification	8	8	0	0
User Login	9	9	0	0
Arena and Court Listing	12	12	0	0
Search and Filter	5	5	0	0
Owner Arena Registration	6	6	0	0
Route Protection	5	5	0	0
Database Integrity	3	3	0	0
Total	**57**	**57**	**0**	**0**

Prepared by the Courtify Sprint 1 team – Shaheer, Yousuf, Zarrar, Wasiq